Virtuoso™ Documentation

High-performance memory-mapped buffer and file access for Delphi

Table of Contents

Overview

Virtuoso is a single-unit, zero-dependency Delphi library that provides unified, thread-safe, generic access to memory-mapped buffers and files. It replaces the need for separate buffer and file-mapping classes by offering one class - `TVirtuoso<T>` - with four operating modes, concurrent read access via MREW (Multi-Read Exclusive-Write) locking, full TStream interoperability, zero-copy sub-views, automatic buffer growth, typed enumeration, and a clean error model.

Why use Virtuoso?

- **One class, four modes.** Anonymous buffers, read-only file views, read-write file views, and copy-on-write - all through the same API. No decision fatigue.
- **Thread-safe by default.** MREW locking means multiple threads can read concurrently. Only writes serialize. And you can turn it off entirely for single-threaded hot paths.
- **Typed generic access.** `TVirtuoso<Single>`, `TVirtuoso<Integer>`, `TVirtuoso<TMyRecord>` - the indexer returns your type directly. No casting, no pointer math.
- **TStream interop.** Call `CreateStream()` and hand the result to any Delphi code that consumes TStream - JSON parsers, image decoders, compression libraries - operating directly over mapped memory with zero copying.
- **Zero-copy sub-views.** Slice a region of the mapping into a lightweight view object. No new OS mapping, no allocation - just pointer arithmetic with bounds checking.
- **Single unit, zero dependencies.** Drop `Virtuoso.pas` into your project. It uses only standard RTL and WinAPI units.

Requirements: Delphi (Win64 recommended for large files). Uses `WinApi.Windows`, `System.SysUtils`, `System.IOUtils`, `System.Classes`, `System.SyncObjs`.

Installation

1. Copy `Virtuoso.pas` into your project's source path.
2. Add `Virtuoso` to your unit's `uses` clause.

That's it. No packages, no components, no third-party dependencies.

```
uses
  Virtuoso;
```

Quick Start

Create an anonymous buffer and write data

```
var
  LBuf: TVirtuoso<Integer>;
begin
  LBuf := TVirtuoso<Integer>.Create();
  try
    if not LBuf.Allocate(1000) then
    begin
      WriteLn('Failed: ' + LBuf.LastError);
      Exit;
    end;

    LBuf[0] := 42;
    LBuf[1] := 100;
    WriteLn(LBuf[0] + LBuf[1]);  // 142

    LBuf.SaveToFile('data.bin');
  finally
    LBuf.Free();
  end;
end;
```

Memory-map a file for reading

```
var
  LFile: TVirtuoso<Byte>;
  LValue: Byte;
begin
  LFile := TVirtuoso<Byte>.Create();
  try
    if not LFile.Open('C:\Data\large_model.bin', vmReadOnly) then
    begin
      WriteLn('Failed: ' + LFile.LastError);
      Exit;
    end;

    WriteLn('File size: ', LFile.Size, ' bytes');
    WriteLn('Elements: ', LFile.Capacity);

    // Random access - the OS pages data from disk on demand
    LValue := LFile[0];
    LValue := LFile[LFile.Capacity - 1];

    // Sequential access via cursor
    LFile.Position := 0;
    while not LFile.Eob() do
      LFile.Read(LValue, SizeOf(Byte));
  finally
    LFile.Free();
  end;
end;
```

Iterate with for-in

```
var
  LBuf: TVirtuoso<Single>;
  LValue: Single;
begin
  LBuf := TVirtuoso<Single>.Create();
  try
    LBuf.Allocate(5);
    LBuf[0] := 1.0; LBuf[1] := 2.0; LBuf[2] := 3.0;
    LBuf[3] := 4.0; LBuf[4] := 5.0;

    for LValue in LBuf do
      WriteLn(LValue:0:1);  // 1.0  2.0  3.0  4.0  5.0
  finally
    LBuf.Free();
  end;
end;
```

Concepts

Memory-Mapped I/O

Traditional file I/O involves explicit read/write calls that copy data between the OS kernel and your process's heap. Memory-mapped I/O takes a different approach: the OS maps a file (or an anonymous region of the system page file) directly into your process's virtual address space. You access the data through a pointer as if it were ordinary memory, and the OS handles paging data in and out of physical RAM transparently.

This means:

- **No explicit buffering.** You don't allocate a TMemoryStream and call ReadBuffer. The data is just *there*, accessible by index or pointer.
- **Demand paging.** Only the pages you actually touch are loaded into RAM. A 4 GB file doesn't consume 4 GB of heap - the OS loads 4 KB pages as needed.
- **Kernel-managed caching.** The OS file cache and your mapped view share the same physical pages. No double-buffering.
- **Efficient for random access.** Seeking is just pointer arithmetic. There's no system call involved until you touch a page that isn't resident.

Virtuoso wraps all of this behind a clean, generic, thread-safe Delphi API.

Mapping Modes

Virtuoso supports four modes, selected when you create or open a mapping:

ModeSourceAccessChanges Persist?Use Case `vmAllocate` System page fileRead-WriteNo (memory only)Scratch buffers, staging areas, IPC `vmReadOnly` Disk fileRead-onlyN/ALoading data files, model weights `vmReadWrite` Disk fileRead-WriteYes (to disk)Patching binaries, database files `vmCopyOnWrite` Disk filePrivate writeNo (file untouched)Sandbox, what-if scenarios

You select the mode through either `Allocate()` (for anonymous buffers) or `Open(Filename, Mode)` (for file-backed mappings). The mode is queryable at runtime via the `MappingMode` property and determines which operations are permitted - writing to a `vmReadOnly` mapping raises `EInvalidOperation`.

Generic Typed Access

`TVirtuoso<T>` is generic over any value type `T`. When you write `TVirtuoso<Single>`, the indexer `Item[i]` returns and accepts `Single` directly. `Capacity` tells you how many `T` elements fit in the mapped region. `Size` tells you the total byte count.

This works with any fixed-size type - `Byte`, `Integer`, `Int64`, `Single`, `Double`, packed records, fixed-size arrays. Because access is implemented via `CopyMemory` with `SizeOf(T)`, there are no alignment restrictions on `T` itself.

```
type
  TPixel = packed record
    R, G, B, A: Byte;
  end;

var
  LImage: TVirtuoso<TPixel>;
begin
  LImage := TVirtuoso<TPixel>.Create();
  LImage.Allocate(1920 * 1080);  // 1080p framebuffer
  LImage[0].R := 255;            // first pixel, red channel
end;
```

API Reference

TVirtuosoMode

```
TVirtuosoMode = (
  vmAllocate,    // Anonymous page-file-backed, read-write
  vmReadOnly,    // File-backed, read-only (PAGE_READONLY)
  vmReadWrite,   // File-backed, read-write (PAGE_READWRITE), changes persist
  vmCopyOnWrite  // File-backed, private writes (PAGE_WRITECOPY), file untouched
);
```

TVirtuoso<T>

The main class. One instance represents one memory mapping - either an anonymous buffer or a file-backed view.

Constructor / Destructor

MethodDescription `constructor Create()` Creates an uninitialized instance. You must call `Allocate()` or `Open()` before using it. `destructor Destroy()` Closes the mapping, releases all OS handles, and frees the MREW lock.

Lifecycle Methods

MethodReturnsDescription `Allocate(ASize: UInt64; AMappingName: string = '')Boolean` Creates an anonymous read-write mapping with room for `ASize` elements of type `T`. Sets mode to `vmAllocate`. If `AMappingName` is non-empty, it is used as the mapping name for cross-process sharing via `OpenShared`; otherwise a GUID is generated. Returns `False` on failure (check `LastError`). Safe to call multiple times - releases any prior mapping first. `OpenShared(AMappingName: string; ASize: UInt64; AReadOnly: Boolean = False)Boolean` Attaches to an existing named memory mapping created by another process (or another `TVirtuoso` instance via `Allocate` with a custom name). `ASize` is the expected size in elements of `T`. When `AReadOnly` is `True`, the mapping is opened for read-only access. Returns `False` on failure (check `LastError`). `Open(AFilename: string; AMode: TVirtuosoMode = vmReadOnly)Boolean` Opens a file-backed mapping in the specified mode. `AMode` must be `vmReadOnly`, `vmReadWrite`, or `vmCopyOnWrite` (not `vmAllocate` - use `Allocate()` for that). Returns `False` on failure. Safe to call multiple times. `Close()` -Unmaps the view, closes the mapping handle and file handle. Resets all state. Idempotent - safe to call when not open.

Threading Methods

MethodDescription `BeginRead()` Acquires a shared read lock. Multiple threads can hold this simultaneously. `EndRead()` Releases the shared read lock. Must be paired with `BeginRead()`. `BeginWrite()` Acquires an exclusive write lock. Blocks until all readers and writers release. `EndWrite()` Releases the exclusive write lock. Must be paired with `BeginWrite()`.

Stream Cursor Methods

MethodReturnsDescription `Read(var ABuffer; ACount: UInt64)UInt64` Reads up to `ACount` bytes from the current position into `ABuffer`. Advances position. Returns bytes actually read (may be less at end of buffer). `Read(var ABuffer: TBytes; AOffset, ACount: UInt64)UInt64` Reads into a `TBytes` array at the specified offset. `Write(const ABuffer; ACount: UInt64)UInt64` Writes `ACount` bytes from `ABuffer` at the current position. Advances position. Returns bytes written. Raises `EInvalidOperation` in `vmReadOnly` mode. Triggers auto-grow if enabled. `Write(const ABuffer: TBytes; AOffset, ACount: UInt64)UInt64` Writes from a `TBytes` array at the specified offset. `ReadString()string` Reads a length-prefixed UTF-16 string from the current position. `WriteString(AValue: string)` -Writes a length-prefixed UTF-16 string at the current position. `Seek(AOffset: Int64; AOrigin: TSeekOrigin)UInt64` Sets position using `soBeginning`, `soCurrent`, or `soEnd` semantics. Returns the new position. Raises on out-of-bounds. `Eob()Boolean` Returns `True` when position has reached or exceeded size (end of buffer).

File I/O Methods

MethodReturnsDescription `SaveToStream(AStream: TStream)` -Writes the entire mapped contents to an arbitrary `TStream`. Acquires a read lock. `SaveToFile(AFilename: string)` -Writes the entire mapped contents to a file. Acquires a read lock. `LoadFromFile(AFilename: string)TVirtuoso<T>` **Class method.** Creates a new `TVirtuoso<T>` instance, allocates an anonymous buffer, and loads the file contents into it. Check `HasError()` on the result - on failure, the instance is valid but empty with `LastError` populated. `FlushToDisk()Boolean` Calls `FlushViewOfFile` to force dirty pages to disk. Only meaningful for `vmReadWrite` mode - returns `True` immediately (no-op) for other modes. Returns `False` on OS failure.

Buffer Operations

MethodDescription `ZeroMemory()` Fills the entire mapped region with zeros. Acquires a write lock. `CopyFrom(ASource: Pointer; ASizeBytes: UInt64)` Copies `ASizeBytes` bytes from `ASource` into the mapping starting at offset 0. Raises if `ASizeBytes` exceeds `Size`. `Fill(AValue: T)` Fills every element in the buffer with `AValue`. Acquires a write lock. `Contains(AValue: T)` Returns `True` if any element in the buffer equals `AValue`. Uses `CompareMem` for byte-level comparison. Acquires a read lock. `IndexOf(AValue: T)` Returns the zero-based index of the first element matching `AValue`, or `-1` if not found. Acquires a read lock.

Feature Methods

MethodReturnsDescription `CreateStream()TStream` Returns a `TVirtuosoStream` instance (a TStream descendant) that reads and writes directly over the mapped memory. The stream has its own position cursor independent of the parent's `Position`. **Caller owns and must free the stream.** The parent `TVirtuoso` must outlive the stream. `CreateView(AOffset, ACount: UInt64)TVirtuosoView<T>` Returns a zero-copy sub-view referencing elements `[AOffset .. AOffset+ACount-1]`. The view shares the parent's MREW lock and has its own position cursor. **Caller owns and must free the view.** The parent must outlive the view. Raises if the range exceeds bounds. `Grow(ANewSize: UInt64)Boolean` Re-maps the anonymous region to hold `ANewSize` elements of `T`, preserving existing data. Only valid for `vmAllocate` mode. Returns `False` with error for file-backed mappings. No-op if already big enough. **Invalidates all existing views and streams.** `GetEnumerator()TVirtuosoEnumerator<T>` Returns a typed enumerator for `for..in` support. The enumerator holds a read lock for its entire lifetime (construction to destruction), guaranteeing a consistent snapshot during iteration.

Error Methods

MethodReturnsDescription `HasError()Boolean` Returns `True` if `LastErrorCode` is non-empty. `ClearError()` -Resets `LastError` and `LastErrorCode` to empty strings.

Properties

PropertyTypeAccessDescription `Item[AIndex: UInt64]T` Read/WriteDefault indexer. Typed access to element at `AIndex`. Write raises `EInvalidOperation` in `vmReadOnly` mode. Raises `EArgumentOutOfRangeException` if `AIndex >= Capacity`. Auto-locks per access. `CapacityUInt64` ReadNumber of `T` elements that fit in the mapping (`Size div SizeOf(T)`). `MemoryPointer` ReadRaw pointer to the start of the mapped region. **Use with caution** - threading is the caller's responsibility when accessing this directly. `SizeUInt64` ReadTotal size of the mapping in bytes. `PositionUInt64` Read/WriteCurrent cursor position for stream-style Read/Write operations. Setting a value beyond `Size` raises `EArgumentOutOfRangeException`. `IsOpenBoolean` Read `True` when a mapping is active (Memory is non-nil). `IsReadOnlyBoolean` Read `True` when mode is `vmReadOnly` or when attached as a read-only shared consumer via `OpenShared`. `IsSharedOwnerBoolean` Read `True` when a mapping is open and was created by this instance via `Allocate()`. `False` when attached via `OpenShared()`. Useful for knowing whether `Close()` will destroy the mapping or just detach. `MappingModeTVirtuosoMode` ReadThe current operating mode. `Filenamestring` ReadThe filename for file-backed mappings. Empty for anonymous buffers. `MappingHandleTHandle` ReadThe OS file mapping handle. Exposed for advanced interop scenarios (e.g., sharing with another process or passing to a C DLL). `MappingNamestring` ReadThe GUID name of the mapping (anonymous mode). Can be used for cross-process sharing via `OpenFileMapping`. `ThreadSafeBoolean` Read/WriteWhen `True` (default), all operations acquire MREW locks. Set to `False` to skip all locking for single-threaded hot paths. `AutoGrowBoolean` Read/WriteWhen `True`, Write operations that exceed the buffer size automatically grow the buffer. Only effective in `vmAllocate` mode. Default: `False`. `GrowFactorDouble` Read/WriteThe multiplier used when auto-growing. Default: `2.0` (double the size each time). `LastErrorstring` ReadHuman-readable description of the last OS-level failure, or empty if no error. `LastErrorCodestring` ReadMachine-readable error code (e.g., `'VT07'`), or empty if no error.

TVirtuosoView<T>

A lightweight, zero-copy window into a parent `TVirtuoso<T>` mapping. Created by `TVirtuoso<T>.CreateView()`. Shares the parent's MREW lock. Has its own position cursor for stream-style access. Does not own or free the parent - the parent must outlive all views.

Methods

MethodReturnsDescription `Read(var ABuffer; ACount: UInt64)UInt64` Reads bytes from the view's current position. Bounds-checked against the view range. `Write(const ABuffer; ACount: UInt64)UInt64` Writes bytes at the view's current position. Raises `EInvalidOperation` if read-only. `SetPosition(AValue: UInt64)` -Sets the view's position cursor. Raises if out of bounds. `Eob()BooleanTrue` when position has reached the end of the view. `SaveToStream(AStream: TStream)` -Writes the view's contents to an arbitrary `TStream`. `SaveToFile(AFilename: string)` -Writes the view's contents to a file.

Properties

PropertyTypeAccessDescription `Item[AIndex: UInt64]T` Read/WriteTyped access relative to the view's start. `View[0]` is the first element of the view, not the parent. `IsOpenBoolean` Read `True` when the view has valid memory. `IsReadOnlyBoolean` ReadMirrors the parent's read-only state at creation time. `MemoryPointer` ReadRaw pointer to the start of the view's region. `SizeUInt64` ReadByte size of the view. `CapacityUInt64` ReadNumber of `T` elements in the view. `PositionUInt64` Read/WriteThe view's own cursor position.

TVirtuosoStream

A standard `TStream` descendant that operates directly over mapped memory. Created by `TVirtuoso<T>.CreateStream()`. Has its own position cursor independent of the parent. Does not own or free the parent.

Overrides `Read`, `Write`, `Seek`, and `GetSize`. Respects the parent's read-only state - `Write` raises `EInvalidOperation` if the parent was opened as `vmReadOnly`. All operations acquire the appropriate MREW lock.

```
var
  LStream: TStream;
begin
  LStream := LBuf.CreateStream();
  try
    // Use LStream with any TStream-compatible API
    LJsonDoc := TJsonDocument.ParseStream(LStream);
  finally
    LStream.Free();
  end;
end;
```

TVirtuosoEnumerator<T>

The enumerator returned by `TVirtuoso<T>.GetEnumerator()`. You rarely interact with this directly - Delphi's `for..in` syntax uses it automatically.

The enumerator acquires a shared read lock in its constructor and releases it in its destructor. This means the data is guaranteed consistent for the entire duration of the loop - no writer can modify it mid-iteration. The compiler-generated `try..finally` ensures the lock is always released, even if the loop body raises an exception.

Threading

Thread safety is a first-class concern in Virtuoso. The implementation uses `TMultiReadExclusiveWriteSynchronizer` (MREW) - Delphi's built-in reader-writer lock - which allows multiple threads to read simultaneously while ensuring exclusive access for writes.

Automatic Locking

Every public method on `TVirtuoso<T>` acquires the appropriate lock internally:

- **Read operations** (Item getter, `Read()`, `Contains()`, `IndexOf()`, `SaveToFile()`, `FlushToDisk()`) acquire a **shared read lock**. Any number of threads can hold this simultaneously.
- **Write operations** (Item setter, `Write()`, `WriteString()`, `ZeroMemory()`, `Fill()`, `CopyFrom()`, `Close()`, `Grow()`) acquire an **exclusive write lock**. This blocks until all readers and other writers release.

For casual, single-threaded use, you don't need to think about threading at all - everything just works.

For multi-threaded use where each operation is independent, it also just works:

```
// Thread A reads index 5 while Thread B reads index 10.
// Both acquire shared read locks - no blocking.

// Thread C writes to index 0. It waits until A and B finish
// their current reads, then acquires exclusive access.
```

Explicit Lock Scoping

When you need multiple operations to be atomic (no other thread can interleave), use explicit locking:

```
// Atomically read two values that must be consistent with each other
LBuf.BeginRead();
try
  LX := LBuf[0];
  LY := LBuf[1];
  LRatio := LX / LY;  // guaranteed that [0] and [1] weren't
                      // modified between the two reads
finally
  LBuf.EndRead();
end;

// Atomically update multiple values
LBuf.BeginWrite();
try
  LBuf[0] := LNewX;
  LBuf[1] := LNewY;
  LBuf[2] := LNewZ;
```

// All three writes are visible to readers as one atomic update
finally
LBuf.EndWrite();
end;

```text
**Important:** When using explicit locking, the individual operations inside the block still attempt to acquire locks internally. MREW supports re-entrant reads, so `BeginRead` inside an outer `BeginRead` is safe. `BeginWrite` inside an outer `BeginWrite` is also safe (MREW tracks the owning thread). However, calling `BeginWrite` inside a `BeginRead` on the same thread will cause a deadlock - this is a fundamental limitation of reader-writer locks. If you need to read and then write atomically, use `BeginWrite` for the outer scope.

### Disabling Locks for Hot Paths

If you know that only one thread accesses the buffer - or you're managing synchronization externally - you can eliminate lock overhead entirely:

```delphi
LBuf.ThreadSafe := False;

// All BeginRead/EndRead/BeginWrite/EndWrite calls become no-ops.
// Item[], Read(), Write(), etc. run with zero lock overhead.
for LI := 0 to LBuf.Capacity - 1 do
  LBuf[LI] := LBuf[LI] * 2;  // tight loop, no lock per iteration

LBuf.ThreadSafe := True;  // re-enable for shared access
```
```

The MREW object is still allocated - `ThreadSafe` just controls whether it's entered. This means toggling is cheap and safe (though you must ensure no other thread is accessing the buffer when you set it to `False`).

Features In Depth

Anonymous Buffers (vmAllocate)

An anonymous buffer is backed by the system page file, not a disk file. It's the Virtuoso equivalent of `TMemoryStream` - but with typed access, thread safety, and auto-grow.

```
var
  LBuf: TVirtuoso<Double>;
begin
  LBuf := TVirtuoso<Double>.Create();
  try
    if not LBuf.Allocate(1000000) then  // 1M doubles = 8 MB
    begin
      WriteLn(LBuf.LastError);
      Exit;
    end;

    // Typed random access
    LBuf[0] := 3.14159;
    LBuf[999999] := 2.71828;

    // Stream-style sequential access
    LBuf.Position := 0;
    LBuf.Write(LMyData, SizeOf(LMyData));

    // Save to disk when needed
    LBuf.SaveToFile('output.bin');

    WriteLn('Mode: ', Ord(LBuf.MappingMode));  // 0 = vmAllocate
    WriteLn('Size: ', LBuf.Size);               // 8000000
    WriteLn('Capacity: ', LBuf.Capacity);       // 1000000
  finally
    LBuf.Free();
  end;
end;
```

The anonymous buffer gets a GUID-based mapping name by default (accessible via `MappingName`). You can also supply a custom mapping name to `Allocate()` for cross-process sharing - see IPC / Shared Memory for details.

Read-Only File Mapping (vmReadOnly)

Maps an existing file for read-only access. The OS pages data from disk on demand - you can map a 10 GB file on a machine with 4 GB of RAM, and only the pages you actually touch consume physical memory.

```
var
  LFile: TVirtuoso<Byte>;
  LHeader: array[0..3] of Byte;
begin
  LFile := TVirtuoso<Byte>.Create();
  try
    if not LFile.Open('model_weights.bin', vmReadOnly) then
    begin
      WriteLn(LFile.LastError);
      Exit;
    end;

    // Random access - OS pages in the needed 4KB page
    WriteLn('First byte: ', LFile[0]);
    WriteLn('Last byte: ', LFile[LFile.Capacity - 1]);

    // Read a header struct
    LFile.Position := 0;
    LFile.Read(LHeader, SizeOf(LHeader));

    // Attempting to write raises EInvalidOperation
    // LFile[0] := $FF;  // <- this would raise
  finally
    LFile.Free();
  end;
end;
```

Read-Write File Mapping (vmReadWrite)

Maps a file with read-write access. Changes to the mapped memory are automatically written back to the file by the OS when pages are flushed. You can force an immediate flush with `FlushToDisk()`.

```
var
  LFile: TVirtuoso<Byte>;
begin
  LFile := TVirtuoso<Byte>.Create();
  try
    if not LFile.Open('config.dat', vmReadWrite) then
      Exit;

    // Modify the file in place - no read/modify/write cycle
    LFile[0] := $FF;
    LFile[1] := $FE;

    // Force changes to disk immediately
    if not LFile.FlushToDisk() then
      WriteLn('Flush failed: ' + LFile.LastError);
  finally
    LFile.Free();  // OS also flushes on unmap
  end;
end;
```

This is useful for large database-style files, binary configuration files, or any scenario where you want to patch a file in place without reading it entirely into a heap buffer, modifying it, and writing it back.

Copy-on-Write File Mapping (vmCopyOnWrite)

Maps a file for reading, but any writes go to a private copy of the affected pages in memory. The original file on disk is never modified. This uses the Windows `PAGE_WRITECOPY` protection.

```
var
  LFile: TVirtuoso<Byte>;
begin
  LFile := TVirtuoso<Byte>.Create();
  try
    if not LFile.Open('original.bin', vmCopyOnWrite) then
      Exit;

    // This write goes to a private copy - original.bin is untouched
    LFile[0] := $FF;

    // You can save the modified version to a new file
    LFile.SaveToFile('modified.bin');
    // original.bin still has its original contents
  finally
    LFile.Free();
  end;
end;
```

This is ideal for "what-if" scenarios: load a binary, apply speculative changes, and decide later whether to persist them. Only the pages you actually modify consume additional memory - unchanged pages share the file's existing cache.

Stream Cursor

Every `TVirtuoso<T>` instance has a position cursor (`Position`) for sequential access, just like `TStream`. The cursor is independent of the typed indexer - you can mix random access via `Item[i]` with sequential access via `Read()` / `Write()`.

```
type
  THeader = packed record
    Magic: UInt32;
    Version: UInt16;
    EntryCount: UInt32;
  end;

var
  LBuf: TVirtuoso<Byte>;
  LHdr: THeader;
begin
  LBuf := TVirtuoso<Byte>.Create();
  try
    LBuf.Open('data.bin', vmReadOnly);

    // Read a fixed-size header
    LBuf.Position := 0;
    LBuf.Read(LHdr, SizeOf(THeader));

    // Seek relative to current position
    LBuf.Seek(10, soCurrent);   // skip 10 bytes

    // Seek from end
    LBuf.Seek(-4, soEnd);       // last 4 bytes

    // Check for end of buffer
    while not LBuf.Eob() do
    begin
      LBuf.Read(LByte, 1);
      // process byte
    end;
  finally
    LBuf.Free();
  end;
end;
```

String Serialization

`WriteString` writes a length-prefixed UTF-16 string (8-byte length prefix followed by the character data). `ReadString` reads it back. This is a binary serialization format - not human-readable text.

```
var
  LBuf: TVirtuoso<Byte>;
  LName: string;
  LCity: string;
begin
  LBuf := TVirtuoso<Byte>.Create();
  try
    LBuf.Allocate(1024);

    LBuf.WriteString('Alice');
    LBuf.WriteString('Wonderland');

    LBuf.Position := 0;
    LName := LBuf.ReadString();  // 'Alice'
    LCity := LBuf.ReadString();  // 'Wonderland'
  finally
    LBuf.Free();
  end;
end;
```

TStream Adapter

`CreateStream()` returns a `TVirtuosoStream` - a full `TStream` descendant that reads and writes directly over the mapped memory with zero copying. This means any Delphi code that consumes `TStream` works out of the box with Virtuoso data:

```
var
  LBuf: TVirtuoso<Byte>;
  LStream: TStream;
  LReader: TStreamReader;
  LLine: string;
begin
  LBuf := TVirtuoso<Byte>.Create();
  try
    LBuf.Open('data.txt', vmReadOnly);
    LStream := LBuf.CreateStream();
    try
      LReader := TStreamReader.Create(LStream);
      try
        while not LReader.EndOfStream do
        begin
          LLine := LReader.ReadLine();
          WriteLn(LLine);
        end;
      finally
        LReader.Free();
      end;
    finally
      LStream.Free();
    end;
  finally
    LBuf.Free();
  end;
end;
```

Key characteristics:

- The stream has its own position cursor, independent of the parent's `Position`.
- Read/Write/Seek all acquire the appropriate MREW lock.
- Write raises `EInvalidOperation` if the parent is read-only.
- The stream does not own the parent - the parent must outlive the stream.
- Thread-safety mirrors the parent's `ThreadSafe` setting at stream creation time.

Zero-Copy Sub-Views

`CreateView(Offset, Count)` returns a `TVirtuosoView<T>` - a lightweight typed window into a sub-region of the parent mapping. No new OS mapping is created; the view is just pointer arithmetic with bounds checking. This is Virtuoso's equivalent of `Span<T>` in .NET.

```
type
  TFileHeader = packed record
    Magic: UInt32;
    Version: UInt16;
    DataOffset: UInt32;
    DataCount: UInt32;
  end;

var
  LFile: TVirtuoso<Byte>;
  LHeaderView: TVirtuosoView<Byte>;
  LDataView: TVirtuosoView<Byte>;
  LHdr: TFileHeader;
begin
  LFile := TVirtuoso<Byte>.Create();
  try
    LFile.Open('structured.bin', vmReadOnly);

    // View the first 14 bytes as the header region
    LHeaderView := LFile.CreateView(0, SizeOf(TFileHeader));
    try
      LHeaderView.Read(LHdr, SizeOf(TFileHeader));
    finally
      LHeaderView.Free();
    end;

    // View the data payload starting at the declared offset
    LDataView := LFile.CreateView(LHdr.DataOffset, LHdr.DataCount);
    try
      // LDataView[0] is the first byte of the payload
      // LDataView.Capacity = LHdr.DataCount
      // Process payload...
    finally
      LDataView.Free();
    end;
  finally
    LFile.Free();
  end;
end;
```

Key characteristics:

- The view shares the parent's MREW lock - no separate synchronization needed.
- It has its own `Position` cursor for stream-style read/write within its bounds.
- `Item[i]` is relative to the view's start - `View[0]` is the first element of the view, not the parent.
- Read-only if the parent is read-only.
- Caller owns and frees the view. Parent must outlive all views.
- **Warning:** `Grow()` invalidates all existing views. Recreate them after growing.

Auto-Grow

For anonymous buffers (`vmAllocate`), you can enable automatic growth. When a `Write()` operation would exceed the current buffer size, Virtuoso re-maps the buffer with a larger size (controlled by `GrowFactor`) and copies the existing data into the new region. This turns the buffer into a growable arena.

```
var
  LBuf: TVirtuoso<Byte>;
  LChunk: array[0..4095] of Byte;
begin
  LBuf := TVirtuoso<Byte>.Create();
  try
    LBuf.Allocate(1024);          // start with 1 KB
    LBuf.AutoGrow := True;
    LBuf.GrowFactor := 2.0;      // double each time (default)

    // Write 1 MB of data - buffer grows transparently
    while LDataAvailable do
    begin
      LBuf.Write(LChunk, SizeOf(LChunk));
      // Buffer automatically grows when needed
    end;

    WriteLn('Final size: ', LBuf.Size);  // may be larger than 1MB
                                         // due to growth rounding
  finally
    LBuf.Free();
  end;
```

You can also grow manually:

```
// Grow to hold at least 10,000 elements
if not LBuf.Grow(10000) then
  WriteLn('Grow failed: ' + LBuf.LastError);
```

Important notes:

- Auto-grow is only effective in `vmAllocate` mode. It's silently ignored for file-backed mappings (writes that exceed the file size return 0 instead).
- `Grow()` on a file-backed mapping returns `False` with error code `VT12`.
- Growing invalidates all existing views and streams - recreate them after a grow.
- Growth acquires an exclusive write lock for the duration of the remap.
- If the buffer is already large enough, `Grow()` is a no-op that returns `True`.

For-In Enumeration

`TVirtuoso<T>` supports Delphi's `for..in` syntax via `GetEnumerator()`:

```
var
  LBuf: TVirtuoso<Integer>;
  LValue: Integer;
  LSum: Int64;
begin
  LBuf := TVirtuoso<Integer>.Create();
  try
    LBuf.Allocate(5);
    LBuf[0] := 10; LBuf[1] := 20; LBuf[2] := 30;
    LBuf[3] := 40; LBuf[4] := 50;

    LSum := 0;
    for LValue in LBuf do
      Inc(LSum, LValue);

    WriteLn('Sum: ', LSum);  // 150
  finally
    LBuf.Free();
  end;
end;
```

The enumerator acquires a shared read lock for the entire loop. This guarantees that no writer can modify the data mid-iteration. Delphi's compiler-generated `try..finally` ensures the lock is released even if the loop body raises an exception.

Note: Because the read lock is held for the entire loop, long-running iterations will block writers. If your loop body does heavy processing, consider copying data out under the lock and processing it separately.

Search and Fill Operations

```
var
  LBuf: TVirtuoso<Integer>;
  LIdx: Int64;
begin
  LBuf := TVirtuoso<Integer>.Create();
  try
    LBuf.Allocate(1000);

    // Fill every element with a value
    LBuf.Fill(0);        // all zeros
    LBuf.Fill(-1);       // all -1

    // Set a specific value
    LBuf[500] := 42;

    // Search for it
    LIdx := LBuf.IndexOf(42);
    WriteLn('Found at: ', LIdx);        // 500

    WriteLn(LBuf.Contains(42));         // True
    WriteLn(LBuf.Contains(999));        // False

    // Zero out the entire buffer (byte-level fill with 0)
    LBuf.ZeroMemory();
  finally
    LBuf.Free();
  end;
end;
```

`Contains` and `IndexOf` use `CompareMem` for byte-level comparison, which works correctly for all value types - integers, floats, packed records, etc. Both acquire a read lock, so they're safe to call from multiple threads.

`Fill` acquires a write lock and iterates over every element.

File I/O

SaveToFile dumps the entire mapped contents to a file:

```
LBuf.SaveToFile('snapshot.bin');
```

This acquires a read lock and streams the data via `TFileStream`. It works in any mode.

LoadFromFile is a class method that creates a new anonymous buffer and loads file contents into it:

```
var
  LBuf: TVirtuoso<Single>;
begin
  LBuf := TVirtuoso<Single>.LoadFromFile('weights.bin');
  try
    if LBuf.HasError() then
    begin
      WriteLn('Error: ' + LBuf.LastError);
      Exit;
    end;

    // The file is now in an anonymous buffer - the file can be
    // deleted or modified without affecting this buffer.
    WriteLn('Loaded ', LBuf.Capacity, ' elements');
  finally
    LBuf.Free();
  end;
end;
```

Note: `LoadFromFile` verifies that the file size is evenly divisible by `SizeOf(T)`. If it isn't (e.g., loading a 7-byte file as `TVirtuoso<Integer>`), the instance is returned with error code `VT05` and no data.

Flush to Disk

For `vmReadWrite` file mappings, the OS automatically flushes dirty pages to disk eventually - but "eventually" may not be soon enough for your use case. `FlushToDisk()` forces an immediate write-back:

```
LFile.Open('database.dat', vmReadWrite);
LFile[0] := $FF;

// Force this change to disk right now
if not LFile.FlushToDisk() then
  WriteLn('Flush failed: ' + LFile.LastError);
```

For all other modes, `FlushToDisk()` is a no-op that returns `True`.

IPC / Shared Memory

Virtuoso provides first-class support for inter-process shared memory. A **producer** process creates a named anonymous buffer, and one or more **consumer** processes attach to it by name.

Producer: Create a Named Shared Buffer

Pass a custom mapping name to `Allocate()` so that other processes can find it:

```
var
  LShared: TVirtuoso<Integer>;
begin
  LShared := TVirtuoso<Integer>.Create();
  try
    if not LShared.Allocate(1024, 'MyApp_SharedData') then
    begin
      WriteLn('Failed: ' + LShared.LastError);
      Exit;
    end;

    // Write data that consumers will read
    LShared[0] := 42;
    LShared[1] := 100;

    WriteLn('Mapping name: ', LShared.MappingName);  // 'MyApp_SharedData'
    WriteLn('IsSharedOwner: ', LShared.IsSharedOwner);  // True

    // Keep the process alive while consumers need the mapping
    ReadLn;
  finally
    LShared.Free();
  end;
end;
```

If a mapping with that name already exists, `Allocate()` returns `False` with error code `VT19`. This ensures the producer owns the name exclusively. When `AMappingName` is empty (the default), a GUID is generated automatically - fully backward compatible with existing code.

Consumer: Attach to an Existing Mapping

From another process, use `OpenShared()` to attach:

```
var
  LConsumer: TVirtuoso<Integer>;
begin
  LConsumer := TVirtuoso<Integer>.Create();
  try
    if not LConsumer.OpenShared('MyApp_SharedData', 1024) then
    begin
      WriteLn('Failed: ' + LConsumer.LastError);
      Exit;
    end;

    WriteLn('Value 0: ', LConsumer[0]);  // 42
    WriteLn('Value 1: ', LConsumer[1]);  // 100
    WriteLn('IsSharedOwner: ', LConsumer.IsSharedOwner);  // False
  finally
    LConsumer.Free();
  end;
end;
```

For read-only access, pass `True` as the third parameter:

```
LConsumer.OpenShared('MyApp_SharedData', 1024, True);
// LConsumer[0] := 99;  // raises EInvalidOperation or OS access violation
```

Synchronization

Cross-process synchronization is the caller's responsibility. The built-in MREW lock only protects threads within a single process. For multi-process coordination, use a named mutex, semaphore, or other OS synchronization primitive around your reads and writes.

IsSharedOwner

The `IsSharedOwner` property tells you whether this instance created the mapping (producer) or attached to it (consumer). This is useful for knowing whether `Close()` will destroy the underlying mapping object or merely detach from it. The OS reference-counts mapping objects - the shared memory remains valid as long as at least one process holds a handle.

Grow Limitations

`Grow()` is not valid on a shared consumer (it would need to resize a mapping owned by another process). Calling `Grow()` on a consumer returns `False` with error code `VT12`.

On the producer side, `Grow()` works but creates a **new** mapping with a new internal name. All existing consumer views in other processes become stale - they will be reading unmapped or invalid memory. If you need to resize shared memory, you must coordinate with all consumers to detach and reattach after the grow.

Error Handling

Error Model Philosophy

Virtuoso uses a split error model:

Exceptions are raised for programmer errors - things that indicate a bug in the calling code. These should never happen in correct programs and should not be caught for flow control:

- `EArgumentOutOfRangeException` - indexing beyond `Capacity`, setting `Position` beyond `Size`, view range exceeding bounds.
- `EInvalidOperation` - writing to a `vmReadOnly` mapping, passing `vmAllocate` to `Open()`.

Boolean returns + LastError are used for OS-level failures - things that can legitimately happen at runtime and should be handled gracefully:

- File not found, access denied, zero-byte file.
- `CreateFileMapping` or `MapViewOfFile` failure.
- `FlushViewOfFile` failure.
- Grow re-mapping failure.

```
// OS failure - check return value
if not LBuf.Open('maybe_missing.bin', vmReadOnly) then
begin
  // Handle gracefully
  WriteLn('Code: ', LBuf.LastErrorCode);  // e.g., 'VT07'
  WriteLn('Message: ', LBuf.LastError);
  Exit;
end;

// Programmer error - let it raise (or fix the bug)
LValue := LBuf[LBuf.Capacity + 1];  // raises EArgumentOutOfRangeException
```

Error Codes Reference

CodeConstantContextDescription `VT01VT_ERROR_ALLOC_SIZE_ZEROAllocate()` Size is zero - cannot allocate an empty buffer. `VT02VT_ERROR_ALLOC_MAPPINGAllocate()CreateFileMapping` failed for anonymous mapping. Includes OS error code. `VT03VT_ERROR_ALLOC_MAPVIEWAllocate()MapViewOfFile` failed for anonymous mapping. Includes OS error code. `VT04VT_ERROR_ALLOC_EXCEPTIONAllocate()` Unexpected exception during allocation. Includes exception message. `VT05VT_ERROR_LOAD_ALIGNMENTLoadFromFile()` File size is not evenly divisible by `SizeOf(T)`. `VT06VT_ERROR_LOAD_EXCEPTIONLoadFromFile()` Exception during file loading (file not found, access denied, etc.). `VT07VT_ERROR_OPEN_FAILEDOpen()CreateFile` failed. Includes filename and OS error code. `VT08VT_ERROR_OPEN_MAPPINGOpen()CreateFileMapping` failed for file. Includes filename and OS error code. `VT09VT_ERROR_OPEN_MAPVIEWOpen()MapViewOfFile` failed for file. Includes filename and OS error code. `VT10VT_ERROR_OPEN_EXCEPTIONOpen()` Unexpected exception during file open. Includes exception message. `VT11VT_ERROR_OPEN_EMPTYOpen()` File is zero bytes - cannot memory-map empty files on Windows. `VT12VT_ERROR_GROW_NOT_ANONYMOUSGrow()` Attempted to grow a file-backed mapping. Only anonymous buffers can grow. `VT13VT_ERROR_GROW_REMAPGrow()` Re-mapping failed during grow. Existing data is preserved. `VT14VT_ERROR_FLUSH_FAILEDFlushToDisk()FlushViewOfFile` returned failure. Includes OS error code. `VT15VT_ERROR_SHARED_NAME_EMPTYOpenShared()` Mapping name is empty - must provide a non-empty name to attach to. `VT16VT_ERROR_SHARED_OPEN_FAILEDOpenShared()OpenFileMapping` failed - mapping not found or access denied. Includes OS error code. `VT17VT_ERROR_SHARED_MAPVIEWOpenShared()MapViewOfFile` failed for the shared mapping. Includes OS error code. `VT18VT_ERROR_SHARED_EXCEPTIONOpenShared()` Unexpected exception during shared open. Includes exception message. `VT19VT_ERROR_ALLOC_NAME_EXISTSAllocate()` A mapping with the specified custom name already exists. The producer should own the name exclusively.

Lifetime and Ownership Rules

Virtuoso uses explicit ownership - the caller is responsible for freeing all objects:

1. `TVirtuoso<T>` - You create it, you free it. The destructor calls `Close()` automatically.
2. `TVirtuosoStream` (from `CreateStream()`) - **Caller owns and must free the stream.** The stream does not own or free the parent `TVirtuoso`. The parent must outlive the stream.
3. `TVirtuosoView<T>` (from `CreateView()`) - **Caller owns and must free the view.** The view does not own or free the parent. The parent must outlive all views.
4. `TVirtuosoEnumerator<T>` (from `GetEnumerator()`) - Managed automatically by Delphi's `for..in` compiler machinery. You don't free it yourself.
5. `Grow()` **invalidation** - When you call `Grow()` (or auto-grow triggers), the underlying memory pointer changes. Any existing views or streams that were created before the grow now hold stale pointers and must not be used. Free them and recreate after growing.

```
// CORRECT lifetime management
LBuf := TVirtuoso<Byte>.Create();
try
  LBuf.Allocate(1024);

  LStream := LBuf.CreateStream();
  try
    LView := LBuf.CreateView(0, 512);
    try
      // Use stream and view...
    finally
      LView.Free();     // free view first
    end;
  finally
    LStream.Free();     // then stream
  end;
finally
  LBuf.Free();          // parent last
end;
```

Practical Examples

Example: Multi-Threaded Image Processing

```
type
  TPixel = packed record
    R, G, B, A: Byte;
  end;

procedure ProcessImageRegion(
  const ABuf: TVirtuoso<TPixel>;
```

text

text

const AStart: UInt64;
const ACount: UInt64);
var
LI: UInt64;
LPixel: TPixel;
begin
// Each thread processes its own range.
// Item[] auto-acquires MREW locks per access.
LI := AStart;
while LI < AStart + ACount do
begin
LPixel := ABuf[LI];

```
// Invert colors
LPixel.R := 255 - LPixel.R;
LPixel.G := 255 - LPixel.G;
LPixel.B := 255 - LPixel.B;

ABuf[LI] := LPixel;
Inc(LI);
```

text

end;
end;

```
// Main thread:
var
LImage: TVirtuoso;
LQuarter: UInt64;
begin
LImage := TVirtuoso.Create();
LImage.Open('photo.raw', vmCopyOnWrite);

// Launch 4 threads, each processing 1/4 of the image
LQuarter := LImage.Capacity div 4;
TTask.Run(procedure begin
ProcessImageRegion(LImage, 0, LQuarter);
end);
TTask.Run(procedure begin
ProcessImageRegion(LImage, LQuarter, LQuarter);
end);
TTask.Run(procedure begin
ProcessImageRegion(LImage, LQuarter * 2, LQuarter);
end);
TTask.Run(procedure begin
ProcessImageRegion(LImage, LQuarter * 3, LQuarter);
end);

// Wait for tasks, then save
LImage.SaveToFile('inverted.raw');
LImage.Free();
end;
```

text

```
### Example: Growable Log Buffer

```delphi
var
  LLog: TVirtuoso<Byte>;
begin
  LLog := TVirtuoso<Byte>.Create();
  try
    LLog.Allocate(4096);          // start small
    LLog.AutoGrow := True;
    LLog.GrowFactor := 1.5;      // grow by 50% each time

    // Write log entries as length-prefixed strings
    LLog.WriteString(FormatDateTime('yyyy-mm-dd hh:nn:ss', Now()) +
      ' Application started');
    LLog.WriteString(FormatDateTime('yyyy-mm-dd hh:nn:ss', Now()) +
      ' User logged in');
    // ... buffer grows as needed ...

    // Read back all entries
    LLog.Position := 0;
    while not LLog.Eob() do
      WriteLn(LLog.ReadString());

    LLog.SaveToFile('session.log');
  finally
```

LLog.Free();
```

text

```
end; end;
```

Example: Binary File Format with Sub-Views

```
type
  TChunkHeader = packed record
    ChunkID: UInt32;
    ChunkSize: UInt32;
  end;

var
  LFile: TVirtuoso<Byte>;
  LHdr: TChunkHeader;
  LChunkView: TVirtuosoView<Byte>;
begin
  LFile := TVirtuoso<Byte>.Create();
  try
    LFile.Open('container.bin', vmReadOnly);

    LFile.Position := 0;
    while not LFile.Eob() do
    begin
      LFile.Read(LHdr, SizeOf(TChunkHeader));

      // Create a view for just this chunk's data
      LChunkView := LFile.CreateView(
        LFile.Position, LHdr.ChunkSize);
      try
        ProcessChunk(LHdr.ChunkID, LChunkView);
      finally
        LChunkView.Free();
      end;

      // Advance past this chunk's data
      LFile.Seek(LHdr.ChunkSize, soCurrent);
    end;
  finally
    LFile.Free();
  end;
end;
```

Example: Passing Mapped Data to a JSON Parser via TStream

```
uses
  System.JSON.Readers,
  System.JSON.Types;

var
  LBuf: TVirtuoso<Byte>;
  LStream: TStream;
  LReader: TJsonTextReader;
  LStreamReader: TStreamReader;
begin
  LBuf := TVirtuoso<Byte>.Create();
  try
    LBuf.Open('config.json', vmReadOnly);
    LStream := LBuf.CreateStream();
    try
      LStreamReader := TStreamReader.Create(LStream);
      try
        LReader := TJsonTextReader.Create(LStreamReader);
        try
          while LReader.Read() do
          begin
            if LReader.TokenType = TJsonToken.PropertyName then
              Write(LReader.Value.ToString + ': ')
            else if LReader.TokenType = TJsonToken.String then
              WriteLn(LReader.Value.ToString);
          end;
        finally
          LReader.Free();
        end;
      finally
        LStreamReader.Free();
      end;
    finally
      LStream.Free();
    end;
  finally
    LBuf.Free();
  end;
end;
```

Performance Notes

Memory-mapped I/O vs TFileStream: For sequential access to small files, `TFileStream` is fine. The advantage of memory mapping appears with large files, random access patterns, and multi-threaded reads. The OS handles caching, and multiple threads can read concurrently without any user-space locking overhead (assuming `ThreadSafe` is on, reads don't block each other).

MREW lock overhead: The `TMultiReadExclusiveWriteSynchronizer` is very fast for the read path - it's essentially an interlocked counter increment/decrement. The write path is heavier because it must wait for all readers to drain. For tight loops doing millions of single-element accesses, the per-access lock overhead can add up. Two strategies:

1. Set `ThreadSafe := False` for the duration of the loop if you know no other thread is accessing the buffer.
2. Use `BeginRead` / `EndRead` (or `BeginWrite` / `EndWrite`) around the entire loop and access `Memory` directly via pointer arithmetic inside the block.

Item[] vs Raw Pointer: The `Item[i]` indexer calls `CopyMemory` for each access, which is a function call plus memory copy. For bulk operations on millions of elements, accessing `Memory` directly (within a `BeginRead` / `EndRead` block) is faster. The indexer is designed for convenience and safety; raw pointer access is available for performance-critical inner loops.

Auto-grow cost: Each grow operation allocates a new mapping, copies all existing data, and frees the old mapping. This is an O(n) operation. If you know the final size in advance, pre-allocate with `Allocate(finalSize)` instead of relying on auto-grow. If you must auto-grow, a `GrowFactor` of 2.0 gives amortized O(1) per write (standard doubling strategy).

Copy-on-Write memory cost: In `vmCopyOnWrite` mode, unchanged pages share the file's cache - the memory cost is nearly zero. Each page you modify (4 KB on most systems) gets a private copy. If you modify the entire file, you'll consume memory equal to the file size. If you only modify a few pages, memory usage is proportional to the modified region.

64-bit recommended: On Win32, the virtual address space is limited to ~2 GB (or ~3 GB with LARGEADDRESSAWARE). Memory-mapping a 1 GB file consumes 1 GB of address space. On Win64, you have 128 TB of virtual address space - map as many files as you want.

Virtuoso VFS

VFS Overview

`Virtuoso.VFS` is a read-only virtual file system built entirely on top of `TVirtuoso<Byte>`. It packs multiple files into a single `.vpk` container archive at build/ship time, then provides instant memory-mapped access to any entry at runtime via `TVirtuosoView<Byte>` - zero-copy, zero-parsing, O(1) lookup.

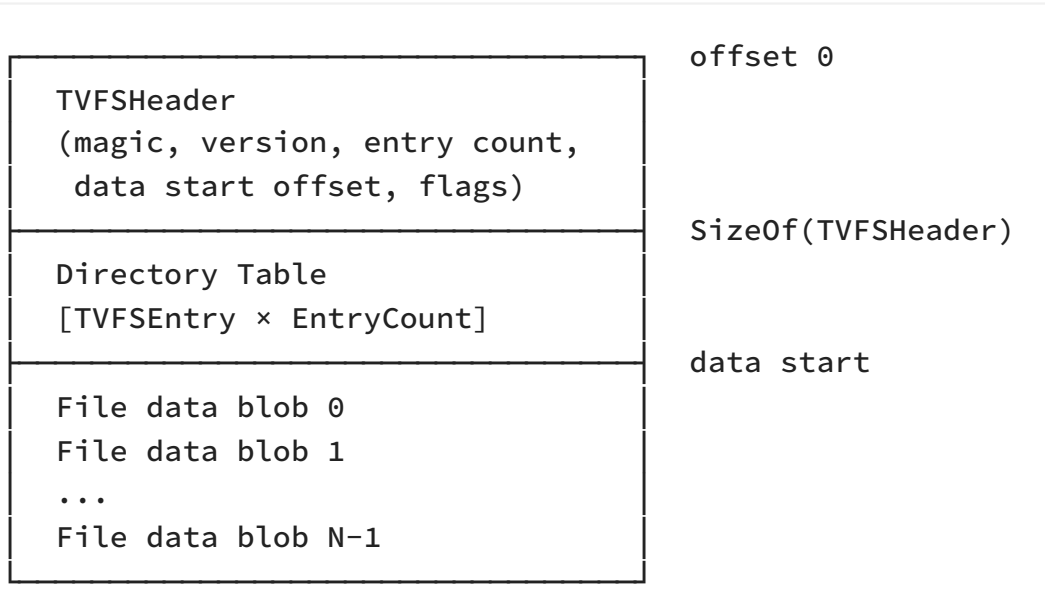
Use case: Game and application asset packaging. All resources (textures, sounds, configs, shaders) are packed once, then opened read-only at runtime. The OS pages data from disk on demand - only the assets you actually touch consume physical memory.

Key properties:

- **Immutable archives.** No add, remove, or update - pack once, read forever. This simplifies everything.
- **Zero-copy access.** `OpenFile()` returns a `TVirtuosoView<Byte>` pointing directly into the memory-mapped archive. No allocation, no copying.
- **O(1) lookup.** An internal `TDictionary<string, Integer>` maps entry paths to directory indices.
- **Case-insensitive paths.** All paths are lowercased on pack and on lookup. `Textures/Sky.PNG` and `textures/sky.png` resolve to the same entry.
- **Single-pass packing.** All file sizes are known up front, so the directory goes at the front and everything is written in one pass.
- **Virtuoso end-to-end.** Both packing and reading use `TVirtuoso<Byte>` - anonymous allocation for building, read-only file mapping for reading.

Dependencies: `Virtuoso.pas`, `System.SysUtils`, `System.IOUtils`, `System.Generics.Collections`.

Container Layout



The directory is at the front because all file sizes are known before packing begins. Each `TVFSEntry` is a fixed-size packed record - no variable-length fields, no parsing required. The data blobs are packed contiguously after the directory with no padding or alignment gaps.

VFS Installation

1. Ensure `Virtuoso.pas` is in your project's source path.
2. Copy `Virtuoso.VFS.pas` alongside it.
3. Add `Virtuoso.VFS` to your unit's `uses` clause.

```
uses
  Virtuoso,
  Virtuoso.VFS;
```

VFS Quick Start

Pack a directory into an archive

```
begin
  if TVirtuosoVFS.PackDirectory('C:\MyGame\Assets', 'assets') then
    WriteLn('Pack succeeded - created assets.vpk')
  else
    WriteLn('Pack failed');
end;
```

The output file always gets the `.vpk` extension regardless of what you pass. `'assets'`, `'assets.dat'`, and `'assets.vpk'` all produce `assets.vpk`.

Read files from an archive

```
var
  LVFS: TVirtuosoVFS;
  LView: TVirtuosoView<Byte>;
begin
  LVFS := TVirtuosoVFS.Create();
  try
    if not LVFS.Open('C:\MyGame\assets') then
    begin
      WriteLn('Failed to open archive');
      Exit;
    end;

    WriteLn('Entries: ', LVFS.EntryCount());

    if LVFS.FileExists('textures/sky.png') then
    begin
      LView := LVFS.OpenFile('textures/sky.png');
      try
        WriteLn('Size: ', LView.Size, ' bytes');
        WriteLn('Data at: ', UIntPtr(LView.Memory));
        // Pass LView.Memory and LView.Size to your texture loader
      finally
        LView.Free();
      end;
    end;
```

finally LVFS.Free(); end; end;

```text
### VFS API Reference

#### Constants

ConstantValueDescription`VFS_MAGIC'VPK0'`4-byte file signature at offset 0 of every archive.`VFS_VERSION1`Format version. Archives with a different version are rejected by `Open()`.`VFS_FILE_EXTENSION'vpk'`Canonical file extension. Enforced by both `Open()` and `PackDirectory()`.

#### TVFSHeader

Packed record at offset 0 of the archive. 52 bytes total.

FieldTypeDescription`Magicarray[0..3] of AnsiChar`File signature - must be `'VPK0'`.`VersionUInt32`Format version - must be `1`.`EntryCountUInt32`Number of file entries in the directory.`DataStartOffsetUInt64`Byte offset where file data blobs begin.`Reservedarray[0..31] of Byte`Reserved for future use, zeroed.

#### TVFSEntry

Packed record for each file in the directory. 540 bytes total.

FieldTypeDescription`EntryPatharray[0..259] of Char`Null-terminated relative path using `/` separator, lowercased.`OffsetUInt64`Byte offset of this entry's data from the start of the archive.`EntrySizeUInt64`Byte length of this entry's data.`ChecksumUInt32`CRC32 checksum of the entry's data.`FlagsUInt32`Reserved for future use (compression, encryption, etc.).

Each entry path can hold up to 259 characters. Paths are stored lowercased with `/` as the separator, regardless of the original OS path format. Subdirectory structure is preserved in the path string - `textures/walking/texture01.png` is a valid entry path.

#### TVFSPackStatus

Enum indicating the current phase of a pack operation.

ValueDescription`vpsStarting`Scan complete, about to begin packing. `FileCount` and `TotalBytes` are now known.`vpsFileBegin`About to pack a specific file. `Filename`, `EntryPath`, `FileIndex`, `FileSize` are populated.`vpsFileEnd`Finished packing a specific file. `BytesWritten` is updated.`vpsCompleted`All files packed, archive finalized and saved to disk.`vpsError`Something went wrong. `ErrorMessage` is populated.

#### TVFSPackInfo

Record passed to the pack callback with progress information.

FieldTypeDescription`StatusTVFSPackStatus`Current phase.`Filenamestring`Full source path of the current file.`EntryPathstring`Relative path inside the archive.`FileIndexInteger`1-based index of the current file.`FileCountInteger`Total number of files being packed.`FileSizeUInt64`Size of the current file in bytes.`BytesWrittenUInt64`Running total of bytes written so far.`TotalBytesUInt64`Grand total bytes across all files.`ErrorMessagestring`Populated only when `Status = vpsError`.

#### TVFSPackCallback

```delphi
TVFSPackCallback = reference to procedure(
  const AInfo: TVFSPackInfo;
  var ACancel: Boolean;
  const AUserData: Pointer
);
```
```

The packer initializes `ACancel` to `False` before each invocation. The callback sets `ACancel := True` to abort packing. The name `ACancel` removes ambiguity - `True` means cancel, no guessing.

TVirtuosoVFS

The main VFS class. One instance represents one opened archive.

Constructor / Destructor

MethodDescription `constructor Create()` Creates an uninitialized instance. Call `Open()` before using. `destructor Destroy()` Calls `Close()`, frees the lookup dictionary and internal `TVirtuoso<Byte>`.

Reading Methods

MethodReturnsDescription `Open(AFilename: string)Boolean` Opens a `.vpk` archive for reading. Forces the `.vpk` extension on the filename. Memory-maps the file read-only, validates the header (magic + version), reads the directory, and builds the O(1) lookup dictionary. Returns `False` if the file doesn't exist, is too small, or has an invalid header. `Close()` -Unmaps the archive, clears the directory and lookup. Idempotent. `IsOpen()BooleanTrue` when an archive is successfully opened. `FileExists(APath: string)Boolean` Checks if an entry exists. Case-insensitive, normalizes slashes. `OpenFile(APath: string)TVirtuosoView<Byte>` Returns a zero-copy view into the entry's data region. The view points directly into the memory-mapped archive - no allocation, no copying. **Caller owns and must free the view.** The `TVirtuosoVFS` instance must outlive the view. Raises `EInvalidOperation` if not open. Raises `EFileNotFoundException` if the entry doesn't exist. `FileSize(APath: string)UInt64` Returns the byte size of an entry. Raises if not open or entry not found. `EntryCount()Integer` Number of entries in the archive. Returns 0 if not open. `ListFiles()TArray<string>` Returns all entry paths in directory order. `ListFiles(ADirectory: string)TArray<string>` Returns entry paths that start with the given directory prefix. Normalizes the prefix and ensures it ends with `/`. Case-insensitive. Returns all descendants, not just immediate children.

Packing Methods

MethodReturnsDescription `PackDirectory(ASourceDir, AOutputFile: string; ACallback: TVFSPackCallback = nil; AUserData: Pointer = nil)Boolean` **Class method.** Scans `ASourceDir` recursively, computes the exact archive size, allocates an anonymous `TVirtuoso<Byte>` buffer, copies all file data with CRC32 checksums, and saves to `AOutputFile` (with forced `.vpk` extension). Fires the callback at each phase. Returns `False` if the directory doesn't exist, is empty, or a file fails to open.

Path Handling

All paths inside a `.vpk` archive use `/` as the separator and are stored in lowercase. This happens automatically during packing - you don't need to normalize paths yourself.

When querying (`FileExists`, `OpenFile`, `FileSize`, `ListFiles`), paths are normalized the same way before lookup. This means all of the following resolve to the same entry:

```
LVFS.OpenFile('textures/sky.png');
LVFS.OpenFile('Textures\Sky.PNG');
LVFS.OpenFile('TEXTURES/SKY.PNG');
```

Entry paths support arbitrary subdirectory depth. A file at `assets/textures/walking/frame01.png` on disk becomes `textures/walking/frame01.png` in the archive (relative to the source directory root).

The maximum path length is 259 characters. Paths longer than this are silently truncated during packing.

CRC32 Integrity

Each entry stores a CRC32 checksum computed from the file's contents at pack time. The checksum uses the standard CRC32 polynomial (`$EDB88320`) with a lookup table initialized once at unit startup.

The current implementation computes and stores the checksum but does not verify it on read. This is by design - verification is left to the application layer, which can choose to validate selectively (e.g., only critical assets) or skip validation entirely for maximum performance.

```
// Manual verification example
var
  LView: TVirtuosoView<Byte>;
  LEntry: TVFSEntry;
begin
  LView := LVFS.OpenFile('data/config.json');
  try
    // Application-level integrity check if needed
    // Compare LEntry.Checksum against your own CRC32 computation
  finally
    LView.Free();
  end;
end;
```

VFS Practical Examples

Pack with progress reporting

```
procedure PackCallback(
  const AInfo: TVFSPackInfo;
  var ACancel: Boolean;
  const AUserData: Pointer);
var
  LPercent: Double;
begin
  if AInfo.Status = vpsStarting then
    WriteLn(Format('Packing %d files (%.2f MB)',
      [AInfo.FileCount, AInfo.TotalBytes / (1024 * 1024)]))

  else if AInfo.Status = vpsFileBegin then
  begin
    LPercent := 0;
    if AInfo.TotalBytes > 0 then
      LPercent := (AInfo.BytesWritten / AInfo.TotalBytes) * 100;
    WriteLn(Format('[%d/%d] %.0f%% %s',
      [AInfo.FileIndex, AInfo.FileCount, LPercent, AInfo.EntryPath]));
  end

  else if AInfo.Status = vpsCompleted then
    WriteLn(Format('Done - %.2f MB written',
      [AInfo.BytesWritten / (1024 * 1024)]))

  else if AInfo.Status = vpsError then
  begin
    WriteLn('ERROR: ' + AInfo.ErrorMessage);
    ACancel := True;
  end;
end;

begin
  TVirtuosoVFS.PackDirectory(
    'C:\MyGame\Assets',
    'C:\MyGame\Bin\assets',
    PackCallback);
end;
```

Load a texture from an archive

```
var
  LVFS: TVirtuosoVFS;
  LView: TVirtuosoView<Byte>;
  LStream: TMemoryStream;
begin
  LVFS := TVirtuosoVFS.Create();
  try
    LVFS.Open('assets');

    LView := LVFS.OpenFile('textures/player/idle01.png');
    try
      // Option 1: Pass raw pointer and size to your loader
      LoadTextureFromMemory(LView.Memory, LView.Size);

      // Option 2: Wrap in a TMemoryStream for APIs that need TStream
      LStream := TMemoryStream.Create();
      try
        LStream.WriteBuffer(LView.Memory^, LView.Size);
        LStream.Position := 0;
        LoadTextureFromStream(LStream);
      finally
        LStream.Free();
      end;
    finally
      LView.Free();
    end;
  finally
    LVFS.Free();
  end;
end;
```

List all files under a directory

```
var
  LVFS: TVirtuosoVFS;
  LFiles: TArray<string>;
  LPath: string;
begin
  LVFS := TVirtuosoVFS.Create();
  try
    LVFS.Open('assets');

    // List everything
    LFiles := LVFS.ListFiles();
    for LPath in LFiles do
      WriteLn(LPath);

    WriteLn('---');

    // List only sounds
    LFiles := LVFS.ListFiles('sounds');
    for LPath in LFiles do
      WriteLn(LPath);
    // Output includes: sounds/music/theme.ogg, sounds/sfx/click.wav, etc.
  finally
    LVFS.Free();
  end;
end;
```

Read a JSON config from an archive

```
uses
  System.JSON;

var
  LVFS: TVirtuosoVFS;
  LView: TVirtuosoView<Byte>;
  LBytes: TBytes;
  LJson: TJSONObject;
begin
  LVFS := TVirtuosoVFS.Create();
  try
    LVFS.Open('assets');

    LView := LVFS.OpenFile('config/settings.json');
    try
      // Copy view data to a byte array for JSON parsing
      SetLength(LBytes, LView.Size);
      CopyMemory(@LBytes[0], LView.Memory, LView.Size);
    finally
      LView.Free();
    end;

    LJson := TJSONObject.ParseJSONValue(LBytes, 0) as TJSONObject;
    try
      WriteLn('Resolution: ', LJson.GetValue('resolution').Value);
    finally
      LJson.Free();
    end;
  finally
    LVFS.Free();
  end;
end;
```